

CUSTOMER

Agent OS Lite

Project vision, product goals, and build sequence

The product in one sentence

A lightweight, production-oriented platform for building and operating AI customer-service agents, proven through an advanced e-commerce support agent.

What this document covers

What we are trying to build and why it matters.

The relationship between the platform and the first customer-support agent.

The user experiences and business journeys the product will enable.

The sequence for evolving one useful agent into a reusable Agent OS.

The project goals, scope boundaries, success criteria, and portfolio story.

Product definition - July 2026

Contents

A product-first guide before technical architecture decisions

1	Executive summary	What the product is, who it serves, and why both a platform and reference agent are necessary.
2	The customer problem	Why customer support remains fragmented, repetitive, costly, and difficult to automate safely.
3	The product we are building	The Agent OS, the reference e-commerce agent, and the two ways customers use the platform.
4	The first product experience	The journeys, users, and examples that make the platform concrete and demonstrable.
5	The build sequence	How we move from one working agent to reusable runtime, operating capabilities, SDK, and Studio.
6	Product goals and principles	What success means and the decisions that keep the product useful, trustworthy, and focused.
7	Scope boundaries	What is deliberately excluded from the first version and why restraint improves the product.
8	Success and portfolio story	How the project will be measured and how it should be presented to recruiters and the market.
9	Future evolution	How the product can expand after the first complete version is proven.
Important boundary This document defines the product. Detailed architecture, framework selection, database design, cloud topology, and implementation plans will be decided in the next technical-design phase.		

Executive summary

One platform, one flagship agent, and a deliberate path from application to operating system

Customer Agent OS Lite is a product for companies that want to create reliable AI agents for customer-facing work. It is not only a chatbot and it is not only a generic agent builder. The product combines a reusable platform with one complete customer-support agent that proves the platform can solve real business problems.

The central product decision

We are building software to build and operate agents, and we are using that software to ship one advanced e-commerce customer-support agent. The agent proves the platform; the platform makes the agent reusable.

The product has three visible parts

Customer Agent OS The reusable product foundation. It defines agents, customer journeys, knowledge, business actions, policies, tests, releases, monitoring, and improvement.	E-commerce Support Agent The first complete agent built on the platform. It handles order tracking, returns, refunds, replacements, damaged items, cancellations, and human escalation.
Agent Studio + SDK Two ways to use the product: a visual workspace for support and operations teams, plus developer capabilities for custom integrations and reusable agent behavior.	Operating Experience The product also includes the experiences needed to trust an agent: evaluation results, approvals, conversation review, cost insights, releases, and rollback.

Who the initial product serves

E-commerce and subscription businesses that receive repetitive but operationally complex support requests.

Customer-support leaders who want more issues resolved without sacrificing control or customer trust.

Support operations teams that manage policies, workflows, quality, escalations, and agent performance.

Developers who must connect agents safely to orders, payments, inventory, shipping, CRM, and ticketing systems.

What “Lite” means

“Lite” means focused, buildable, and intentionally smaller than a broad enterprise platform such as Sierra. It does not mean a toy. The product will cover a narrow set of customer journeys deeply, while demonstrating the complete lifecycle required to create, release, observe, and improve production-oriented agents.

2

The customer problem

Customer support is full of repeated work, fragmented context, and unsafe handoffs

Most companies do not suffer from a lack of chat interfaces. They suffer because resolving one customer issue often requires knowledge, judgment, business rules, several internal systems, and a human fallback. Traditional bots can answer questions, but many cannot complete the underlying job.

Customers repeat themselves A customer moves from chat to email or a human representative and must explain the same problem again because context is fragmented.	Agents switch between tools Support representatives copy information across order systems, CRM, shipping portals, billing tools, and ticketing systems.
Knowledge becomes stale Policies and help articles conflict, change over time, and are applied inconsistently by humans and AI systems.	Automation stops at answers Many bots can describe a return policy but cannot verify eligibility, create the return, reserve a replacement, and confirm completion.
Risk is uneven A shipping update is low risk; a refund, identity change, or policy exception needs stricter checks and sometimes human approval.	Failures do not create learning Bad conversations are reviewed manually, but often do not become structured tests that prevent the same mistake in the next release.

The opportunity

The product opportunity is to move from conversation automation to controlled customer-outcome automation. Customer Agent OS Lite should help a business define what an agent can accomplish, connect that agent to approved knowledge and actions, supervise risky decisions, and learn from every failure.

Outcome-first product thesis

The customer should not merely receive a fluent answer. The product should either complete the requested outcome, show clear progress, or hand the case to a human with complete context.

Why begin with e-commerce

The journeys are familiar and easy for recruiters, users, and potential customers to understand.

The workflows include both simple questions and consequential actions.

The product can demonstrate knowledge, tools, approvals, images, asynchronous updates, and handoff without entering heavily regulated medical or financial decisions.

Success is measurable through completed returns, resolved cases, reduced handoffs, response time, customer satisfaction, and cost per resolution.

3

The product we are building

A reusable customer-agent platform with one complete reference application

A. Customer Agent OS - the main product

The Agent OS is the reusable system a company uses to define, deploy, operate, and improve customer-facing agents. It gives an organization one controlled place to manage the agent's purpose, supported journeys, knowledge, actions, policies, human approvals, quality tests, releases, and performance.

Define the agent Set its role, tone, supported languages, customer journeys, boundaries, and intended outcomes.	Connect company knowledge Provide policies, product information, help content, operating procedures, and approved answers.
Connect business actions Allow the agent to check orders, create returns, update subscriptions, open tickets, and request approvals.	Control risk Specify identity requirements, refund limits, restricted actions, escalation conditions, and approval thresholds.
Test and release Simulate customer situations, compare versions, publish an approved release, and roll back when quality falls.	Observe and improve Review outcomes, failures, costs, human overrides, knowledge gaps, and opportunities for the next release.

B. E-commerce Customer Support Agent - the first application

The reference agent is a real customer-facing product built entirely on the Agent OS. It is not a separate hard-coded demo. Its journeys, actions, policies, knowledge, tests, and releases should all be represented through reusable platform capabilities.

C. Agent Studio and SDK - two ways to build

Support and operations teams use Agent Studio to configure journeys, policies, knowledge, tests, and releases. Developers use the SDK and configuration-as-code to add custom actions, integrations, data sources, and advanced behavior. Both experiences manage the same underlying agent definition.

How the parts relate

Product	Customer Agent OS Lite
Reusable platform	Customer Agent OS
First agent	E-commerce Customer Support Agent
Creation experiences	Agent Studio for operations teams + SDK/configuration for developers

Product test

If a second company can define different policies, tools, knowledge, branding, and journeys without rewriting the core runtime, the product is behaving like an Agent OS rather than a single custom bot.

4

The first product experience

A small set of customer journeys, implemented deeply from request to outcome

The first release should not try to support every possible customer-service request. It should offer a carefully selected set of journeys that increase in complexity and collectively prove the product concept.

Journey	Example request	Product outcome
Order tracking	“Where is my order?”	Retrieve the live order and shipment status, explain delays, and give the next expected milestone.
Return eligibility	“Can I return this item?”	Identify the order, apply the correct current policy, explain the decision, and start a return when eligible.
Refund or replacement	“I received the wrong item.”	Verify the issue, offer permitted outcomes, perform the selected action, and confirm completion.
Damaged-item claim	“This arrived broken.”	Collect evidence, inspect an uploaded image, apply risk and policy rules, and create a replacement or escalation.

Journey	Example request	Product outcome
Subscription cancellation	"Cancel my membership."	Explain impact, offer an appropriate retention option when allowed, process cancellation, and confirm final status.
Human escalation	"I need a person."	Transfer the customer with a complete summary, relevant records, attempted actions, risk level, and recommended next step.

Flagship demonstration: damaged replacement to a new address

A customer says: "The headphones I received are damaged. I want a replacement delivered to a different address." The product should guide the customer through identity, order lookup, evidence collection, eligibility, inventory, address validation, risk checks, approval when necessary, replacement creation, return instructions, and proactive follow-up.

1	Understand	Identify the customer's actual goal: replace a damaged item and change the delivery address.
2	Gather context	Find the order, product, customer history, current policies, shipping status, and available inventory.
3	Clarify	Ask only for missing information, such as a photograph or the reason for the new address.
4	Decide safely	Apply the current return, replacement, address-change, and approval rules.
5	Complete actions	Create the return, reserve the replacement, validate the address, and send confirmation.
6	Continue or hand off	Track delayed steps or transfer the case to a human with all context when automation should stop.

Why this example matters

It proves the product can move beyond answering questions. It must coordinate knowledge, customer context, business decisions, actions, waiting states, approvals, and human collaboration while maintaining one coherent customer experience.

4.1

Who experiences the product

Customer Agent OS Lite is not only a customer chat window

Customer Uses chat first, and later email or voice. Receives clear answers, completed actions, progress updates, and a smooth human transfer when needed.	Human support representative Receives escalations with the customer's goal, conversation summary, policies used, actions attempted, errors, and recommended next step.
Support operations manager Defines journeys, updates policies, reviews unresolved conversations, manages approvals, runs tests, and publishes improved releases.	Developer / platform engineer Adds secure business actions and integrations, extends agent behavior, creates reusable capabilities, and validates that the platform works for multiple tenants.
Customer-support leader Tracks resolution, escalation, quality, customer satisfaction, risk, and cost by journey and release.	Product or business leader Discovers recurring customer pain, broken processes, knowledge gaps, and opportunities to remove the reason support was needed.

A complete product journey

A journey begins with a customer request but does not end when the agent sends a message. Customer Agent OS Lite should represent the full lifecycle: define the intended outcome, support the live interaction, complete or supervise business actions, measure the result, learn from failure, test an improvement, and release a better version.

1	Define Objective: A support team creates or updates a customer journey. Example: A failed replacement conversation can become a permanent test case. Result: The product improves as an operating system, not as a collection of one-off prompt edits.
2	Test Objective: The team simulates normal, ambiguous, risky, and adversarial customer situations. Example: A failed replacement conversation can become a permanent test case. Result: The product improves as an operating system, not as a collection of one-off prompt edits.

3

Release

Objective: An approved version is published to a selected customer channel.

Example: A failed replacement conversation can become a permanent test case.

Result: The product improves as an operating system, not as a collection of one-off prompt edits.

4

Resolve

Objective: The agent helps real customers and completes allowed business outcomes.

Example: A failed replacement conversation can become a permanent test case.

Result: The product improves as an operating system, not as a collection of one-off prompt edits.

5

Escalate

Objective: A human takes over when confidence, policy, risk, or customer preference requires it.

Example: A failed replacement conversation can become a permanent test case.

Result: The product improves as an operating system, not as a collection of one-off prompt edits.

6

Learn

Objective: Failures, overrides, costs, and customer feedback become inputs to the next release.

Example: A failed replacement conversation can become a permanent test case.

Result: The product improves as an operating system, not as a collection of one-off prompt edits.

5

The build sequence

Start with one useful agent, then deliberately extract the reusable product

The sequence is essential. Beginning with a generic visual agent builder would produce abstractions without evidence. Building only one hard-coded agent would demonstrate an application but not a platform. The correct path is to solve a real customer problem first and convert each repeated capability into a product primitive.

1

Build the reference customer-support agent

Objective: Deliver a useful e-commerce agent with a small set of complete customer journeys.

Example: The agent tracks an order, starts a return, creates an approved refund, and hands off difficult cases.

Result: We learn the real requirements before creating generic platform concepts.

2

Extract reusable agent-runtime capabilities**Objective:** Separate the general concepts from e-commerce-specific behavior.**Example:** Order lookup remains a commerce-specific action, while journey selection, conversation state, knowledge access, and escalation become reusable capabilities.**Result:** A second agent can be created without duplicating the entire system.

3

Add production operating capabilities**Objective:** Make the agents governable, measurable, releasable, and safe to improve.**Example:** The team can run simulations, inspect failures, require approvals, compare releases, and roll back a bad version.**Result:** The platform becomes an operating product rather than a prompt demo.

4

Expose configuration and a developer SDK**Objective:** Allow teams to create new agents and integrations without changing the platform core.**Example:** A developer adds a shipping integration; an operations manager changes the refund threshold and reruns the relevant tests.**Result:** The product becomes reusable across companies and customer-service domains.

5

Add a focused Agent Studio**Objective:** Give non-engineering teams a visual way to manage the parts they own.**Example:** A support operations manager uploads a new policy, adjusts an escalation condition, reviews test results, and publishes a release.**Result:** The platform becomes usable by the people closest to customer pain.

6

Validate scale, reliability, and portfolio evidence**Objective:** Prove the product's design through benchmarks, failure testing, documentation, and a polished demonstration.**Example:** The demo includes load-test evidence, release comparison, provider failure handling, human approval, and a complete customer outcome.**Result:** The project communicates principal-level product and engineering ownership without overstating real-world traffic.**The extraction rule**

A capability becomes part of the Agent OS only after the reference agent proves why it is needed. This protects the product from premature abstractions and keeps every platform feature connected to a real customer outcome.

5.1

Sequence examples

How a feature evolves from one agent behavior into a reusable product capability

Starting behavior	What we learn	OS capability created	Reusable value
Return policy	The first agent reads one e-commerce return policy.	The OS introduces versioned knowledge sources, effective dates, business areas, access rules, and tests for policy changes.	Another company can supply its own policies without changing the runtime.
Refund action	The first agent calls a commerce API to issue a refund.	The OS introduces approved business actions, typed inputs, permissions, thresholds, confirmation, approvals, and audit history.	Other consequential actions can reuse the same safety model.
Human escalation	The first agent creates a support ticket when it cannot continue.	The OS introduces escalation reasons, queue routing, handoff summaries, pending approvals, and return of control from human to AI.	Every future agent gains a consistent human-collaboration experience.
Failed conversation	A customer discovers that the agent applies an outdated exchange rule.	The OS captures the conversation as an evaluation case, associates it with the affected journey, verifies the fix, and blocks release if it regresses.	Failures become permanent quality assets.
Model choice	The first agent uses one model for every task.	The OS introduces task-specific model policies, quality thresholds, budgets, fallbacks, and release comparisons.	Companies can balance quality, speed, and cost without rewriting agents.

A second-agent test

After the platform supports the e-commerce agent, we should configure a small second agent - for example, a SaaS subscription-support agent. It does not need the same depth. Its purpose is to prove that the platform's journeys, knowledge, actions, policies, tests, releases, and monitoring are genuinely reusable rather than hidden e-commerce assumptions.

Product goals and principles

The standards that should guide every product decision

1	Resolve customer outcomes Objective: Complete the job when safe and authorized; otherwise provide progress or a high-quality handoff. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.
2	Make behavior configurable Objective: Company-specific journeys, policies, knowledge, tools, branding, and approval thresholds should not require rewriting the core platform. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.
3	Keep humans in control Objective: The product must make it easy to review, approve, override, correct, and resume work. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.
4	Create measurable quality Objective: Every release should be compared using customer outcomes, policy adherence, safety, latency, cost, and human feedback. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.
5	Turn failures into improvement Objective: Real mistakes should become evaluation cases, product changes, and release protections. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.
6	Demonstrate enterprise thinking Objective: The project should communicate multi-tenancy, permissions, auditability, safe actions, reliability, and thoughtful scope. Example: Each goal should be visible in the product demo and documentation. Result: The product story and technical implementation reinforce each other.

7

Remain understandable

Objective: A reviewer should understand the customer problem, business value, and product experience before seeing the technical architecture.

Example: Each goal should be visible in the product demo and documentation.

Result: The product story and technical implementation reinforce each other.

8

Be truthful about scale

Objective: The system can be designed and load-tested against a ten-million-user model without claiming ten million real users.

Example: Each goal should be visible in the product demo and documentation.

Result: The product story and technical implementation reinforce each other.

Core product principles

Outcome before conversation: judge the agent by whether the customer's problem is solved.

Bounded autonomy: expand automation only where permissions, policies, and quality evidence support it.

One customer context: preserve the customer's goal and history across AI and human collaboration.

Business-level actions: expose safe, meaningful capabilities rather than unrestricted low-level access.

Release discipline: treat agent behavior as a versioned product that can be tested, approved, compared, and rolled back.

Narrow first, reusable second: solve a limited domain deeply and generalize only what has been proven.

7

Scope boundaries

What the first version deliberately will not attempt

A credible product strategy is defined as much by what it excludes as by what it includes. The first complete version should be ambitious in lifecycle and quality, but narrow in vertical, channels, and supported journeys.

No foundation model

The project will use existing models. Building or training a frontier model would distract from customer-agent systems and product execution.

No universal agent builder

The first Studio will manage customer-service agents and proven journey types, not every possible autonomous workflow.

No full contact center

The product will not reproduce workforce scheduling, telephony infrastructure, every messaging channel, or all existing help-desk features.

No unrestricted agent swarm

Specialized agent behavior will be added only where it improves a measurable journey; complexity is not a product goal.

No all-industry launch E-commerce and subscriptions are the reference domain. Finance, healthcare, insurance, and other regulated verticals are future expansions.	No unsupported compliance claims The product can demonstrate security and governance patterns but will not claim formal certification without completing the required process.
No early voice dependency Chat will prove the product first. Voice can be added later using the same journeys, knowledge, actions, policies, and operating model.	No fake production claims Benchmark targets and simulated workloads will be labeled clearly; real users and real production history will not be invented.

What remains ambitious

Even with these boundaries, the product covers the full journey from agent definition to customer outcome, human collaboration, evaluation, release management, improvement, and portfolio-grade evidence.

The first-version promise

A company should be able to configure an e-commerce customer-support agent, connect its approved knowledge and actions, define risk and approval rules, test realistic customer scenarios, release a version, handle live conversations, collaborate with human representatives, and understand where the agent succeeded or failed.

8

Success and portfolio story

How we will know the product works and what the project should communicate

Product success

Customer outcomes The supported journeys complete correctly, customers receive clear status, and handoffs preserve context.	Operational control A support team can update policies, approvals, tests, and releases without changing the platform core.
Reusable platform A small second agent can be configured with different knowledge, policies, and actions.	Quality evidence The project shows evaluation results, failure examples, corrected releases, and regression protection.

Business visibility

Dashboards communicate resolution, escalation, cost, quality, latency, and customer feedback by journey.

Trustworthy operation

Risky actions require the right identity, permissions, policies, approvals, and audit history.

Portfolio success

The project should make a recruiter or engineering leader conclude that the builder can own an ambiguous AI product end to end: understand the business domain, choose a focused wedge, design reusable platform concepts, build a polished user experience, define quality and safety, reason about scale and reliability, and communicate tradeoffs clearly.

Desired recruiter impression

“This is not a chatbot demo. This engineer designed a reusable customer-agent platform, shipped a complete reference agent, and treated agent quality, human operations, product usability, cost, release management, and scale as one coherent system.”

Recommended public description**Customer Agent OS Lite**

A production-oriented platform for building and operating AI customer-service agents. The first reference agent resolves e-commerce order, return, refund, replacement, damaged-item, cancellation, and escalation journeys using configurable knowledge, business actions, policies, evaluations, human approvals, releases, and operational insights.

Truthful scale language

Use the phrase “designed and load-tested for a documented ten-million-customer traffic model.” Do not use “serves ten million users” unless the project actually has that production history. The evidence should include assumptions, benchmark results, bottlenecks found, and the improvements made.

The artifacts that complete the product story

A live customer journey demo and a human handoff demo.

Agent Studio screens for journeys, knowledge, policies, tests, releases, and insights.

A concise product requirements document and customer-journey definitions.

A polished repository with examples, SDK usage, and a second-agent proof.

Evaluation and load-test reports with honest assumptions and results.

A public article or video explaining the product decisions and sequence from agent to Agent OS.

Future evolution

Where Customer Agent OS Lite can go after the first product is proven

The initial product is a foundation, not the final market boundary. Expansion should follow evidence from the reference agent and should reuse the same product model rather than adding unrelated features.

More customer channels Add email, SMS, WhatsApp, and voice while preserving the same agent identity, journeys, policies, memory, and handoff model.	Proactive customer service Notify customers about delays, failed payments, expiring subscriptions, product issues, or recommended next steps before they contact support.
Knowledge improvement agent Detect missing or conflicting knowledge, draft updates from resolved cases, and verify that new content improves customer outcomes.	Agent improvement assistant Turn failures into proposed journey, policy, knowledge, or evaluation changes for a human to review.
Additional vertical agents Expand to SaaS support, travel disruption, insurance intake, telecom service, or regulated workflows after adding the required domain controls.	Customer intelligence Aggregate support interactions into product bugs, recurring friction, churn reasons, policy problems, and roadmap opportunities.
Partner ecosystem Allow approved developers to publish integrations, business actions, journey templates, evaluation packs, and vertical solutions.	Outcome-based commercial model For a real company, price selected journeys around measurable completed outcomes rather than only seats or messages.

Final product vision

From support agent to customer relationship operating system

Begin with one e-commerce agent that resolves customer problems. Evolve into a reusable platform where companies can create trusted customer-facing agents for support, success, onboarding, retention, and proactive engagement - all governed through the same journeys, knowledge, actions, policies, evaluations, human collaboration, and release lifecycle.

Decision before technical design

Next, we will finalize the first journeys, personas, outcomes, approval boundaries, and product screens. Those decisions will become the product contract for technical design.

**Build one agent that genuinely works.
Then turn everything it repeatedly needs into the
operating system.**